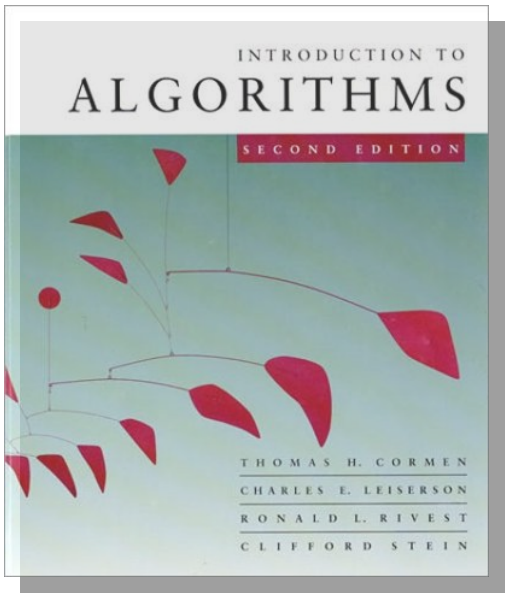
Introduction to Algorithms
6.046J/18.401J



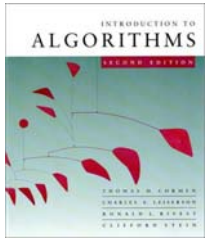
LECTURE 1

Analysis of Algorithms

- Insertion sort
- Asymptotic analysis
- Merge sort
- Recurrences

Prof. Charles E. Leiserson

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Insertion sort analysis

Worst case: Input reverse sorted.

$$T(n) = \sum_{j=2}^{n} \Theta(j) = \Theta(n^2) \qquad \text{[arithmetic series]}$$

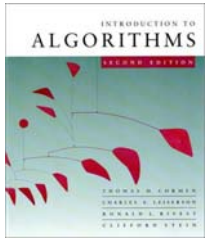
Average case: All permutations equally likely.

$$T(n) = \sum_{j=2}^{n} \Theta(j/2) = \Theta(n^2)$$

Is insertion sort a fast sorting algorithm?

- Moderately so, for small n.
- Not at all, for large n.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Merge sort

MERGE-SORT $A[1 \ldots n]$

1. If $n = 1$, done.
2. Recursively sort $A[1 \ldots \lceil n/2 \rceil]$ and $A[\lceil n/2 \rceil + 1 \ldots n]$.
3. "***Merge***" the 2 sorted lists.

Key subroutine: **MERGE**

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays

20	12
13	11
7	9
2	1

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays

20	12
13	11
7	9
2	1

1

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

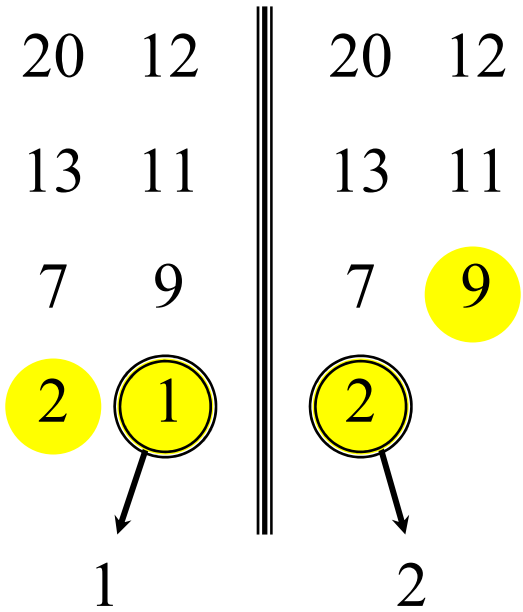
Merging two sorted arrays

20	12	20	12
13	11	13	11
7	9	7	9
2	1	2	

1

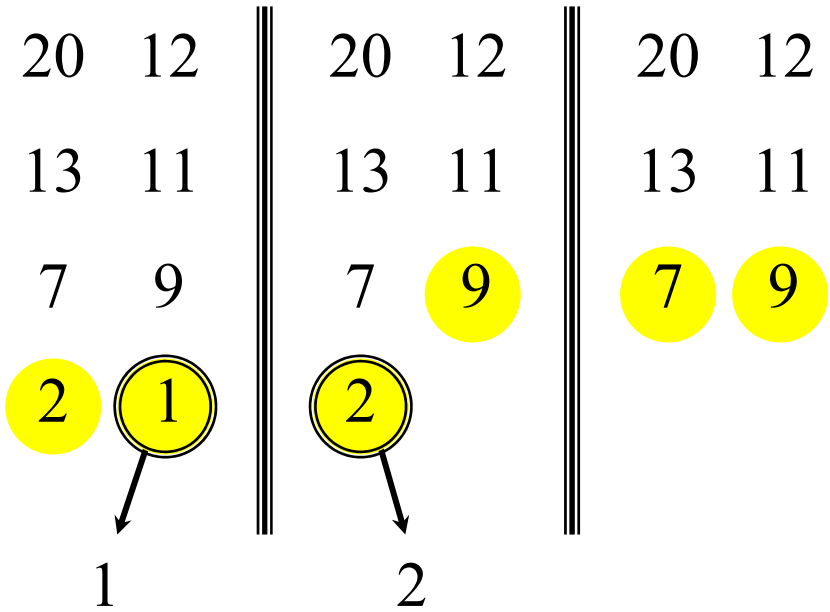
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



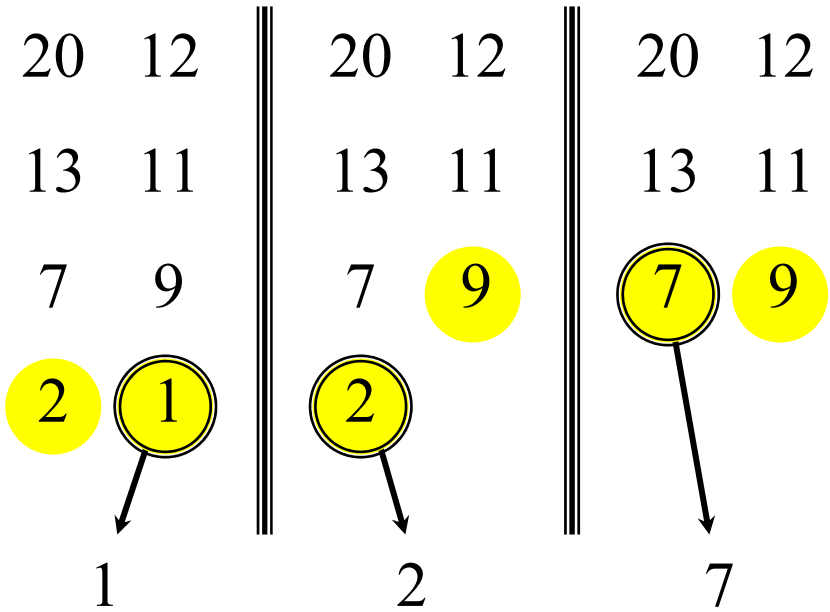
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



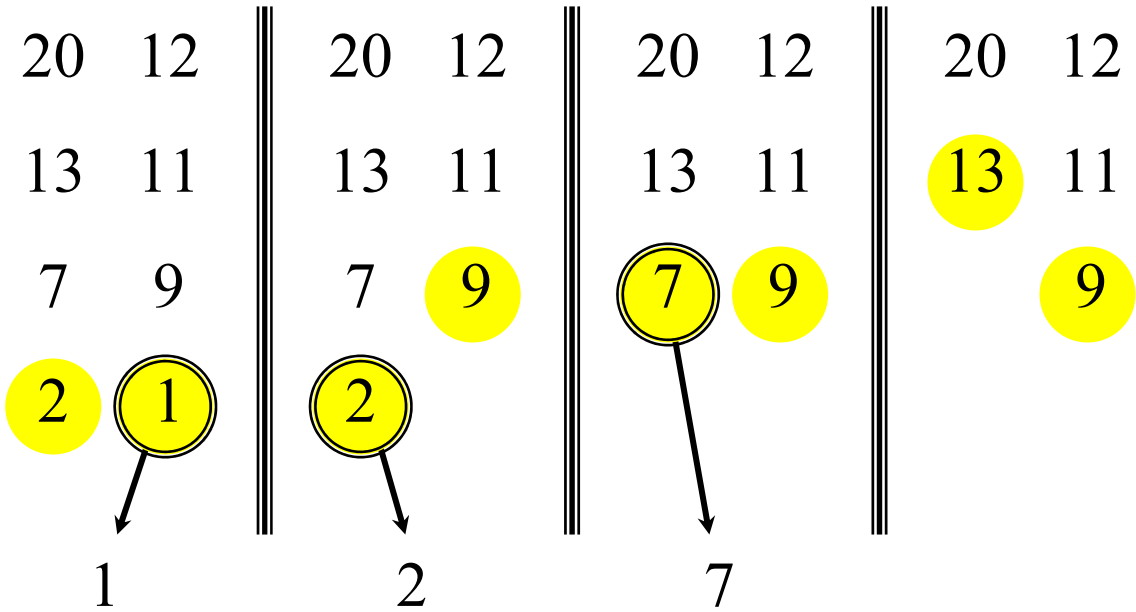
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



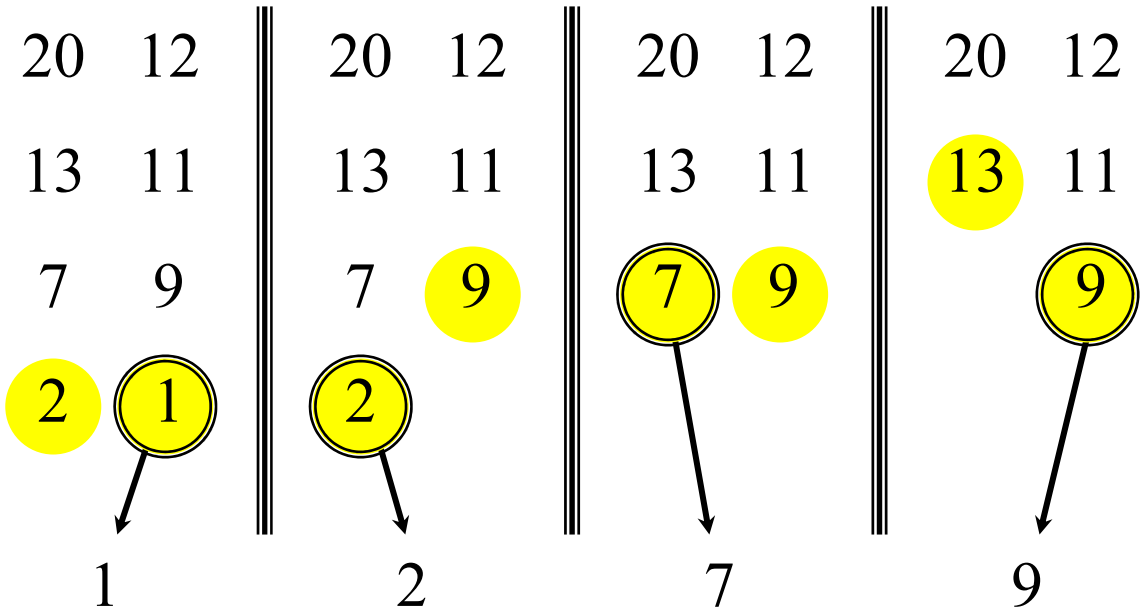
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



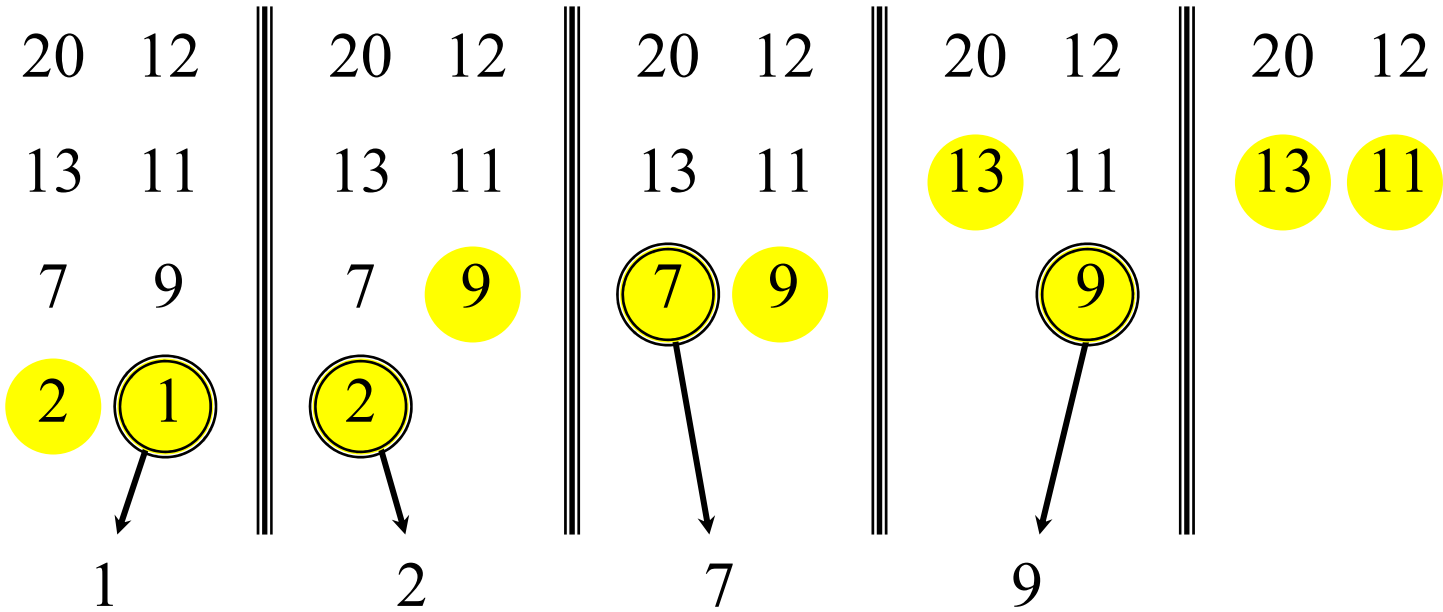
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



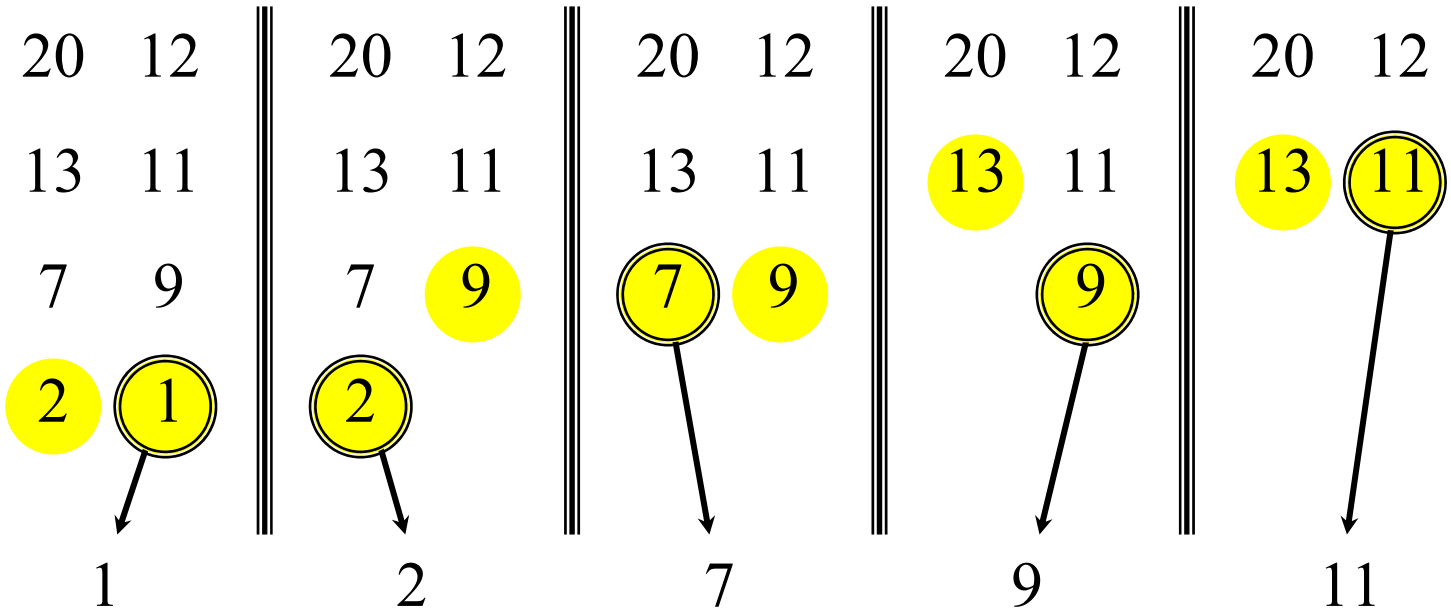
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



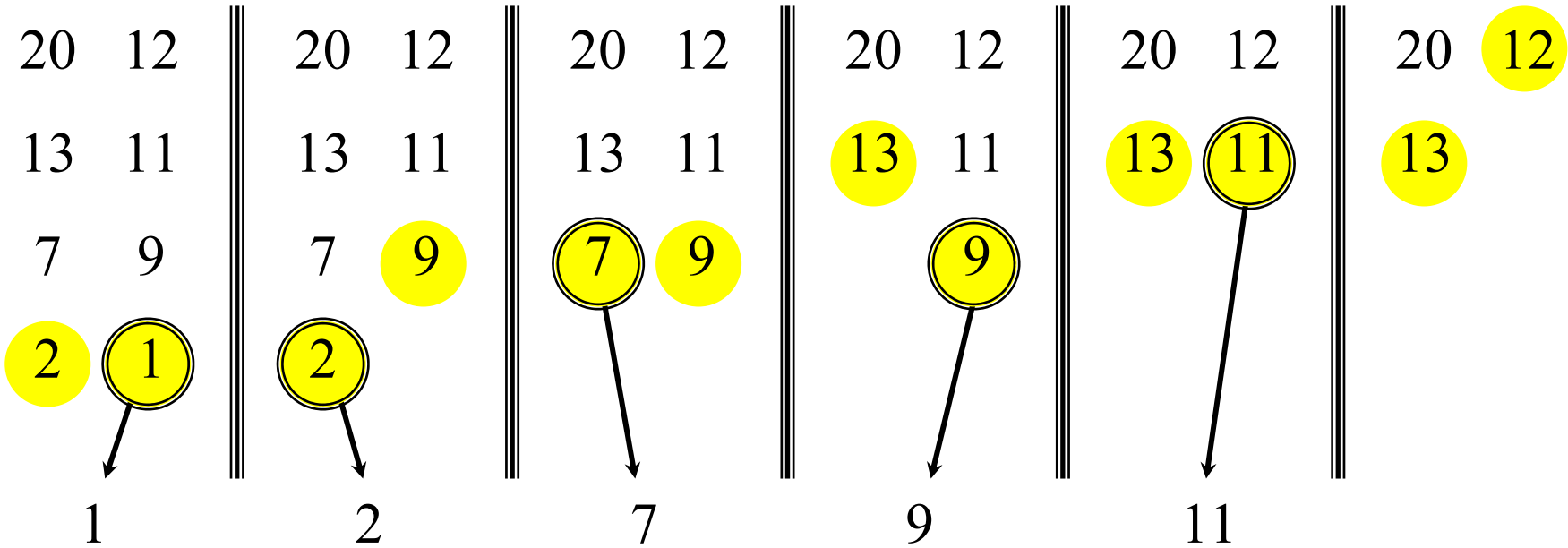
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



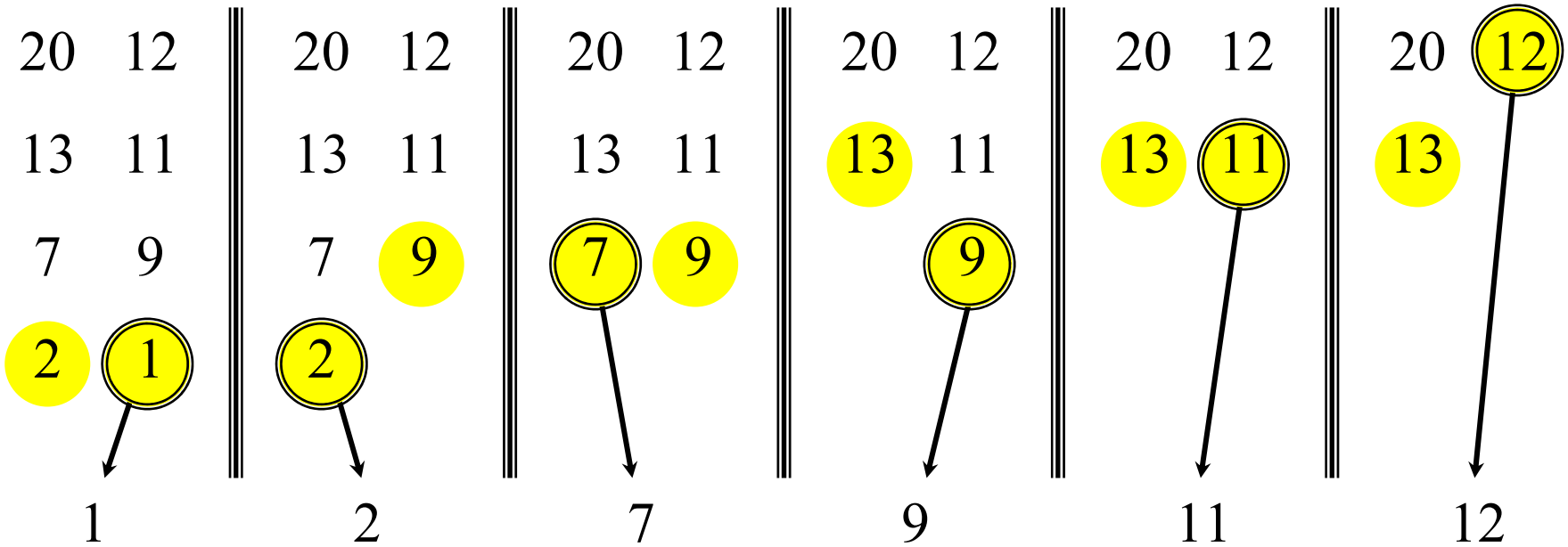
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



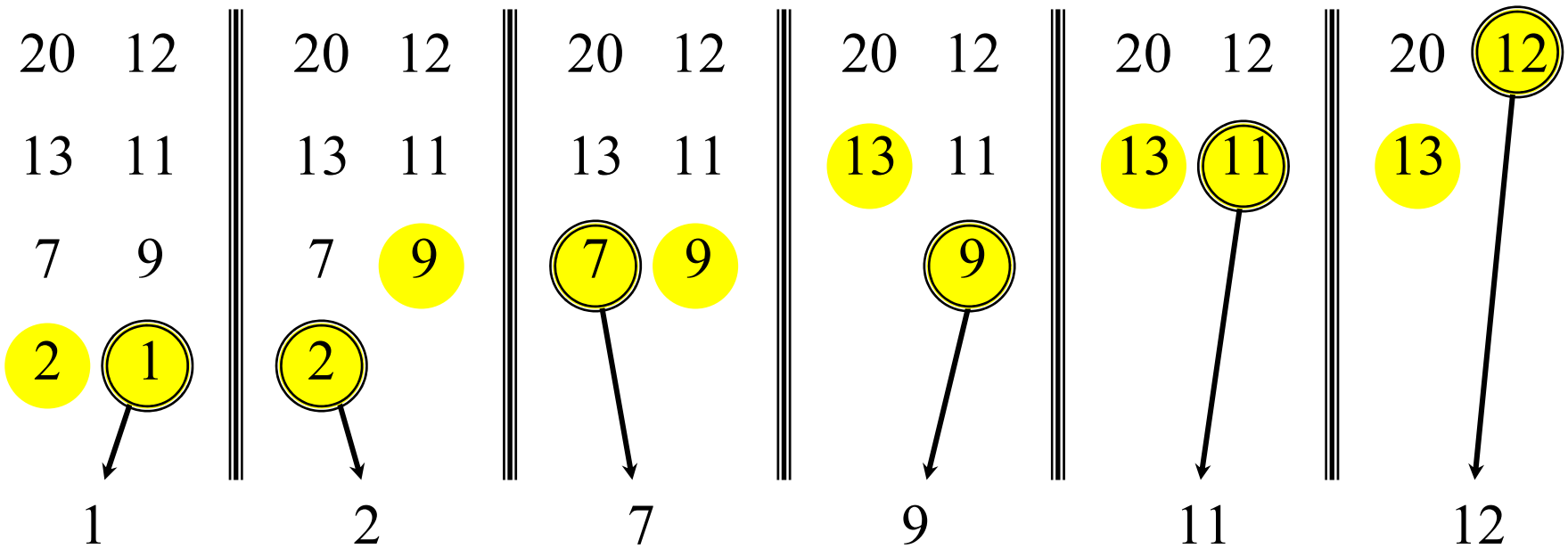
Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Merging two sorted arrays



Time = $\Theta(n)$ to merge a total of n elements (linear time).

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

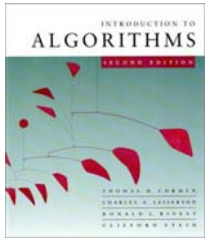
Analyzing merge sort

$T(n)$	**MERGE-SORT** $A[1 \,..\, n]$
$\Theta(1)$	1. If $n = 1$, done.
$2T(n/2)$	2. Recursively sort $A[\,1 \,..\, \lceil n/2 \rceil\,]$ and $A[\,\lceil n/2 \rceil + 1 \,..\, n\,]$.
$\Theta(n)$	***3. "Merge"*** the 2 sorted lists

Abuse

Sloppiness: Should be $T(\lceil n/2 \rceil) + T(\lfloor n/2 \rfloor)$, but it turns out not to matter asymptotically.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson



Recurrence for merge sort

$$T(n) = \begin{cases} \Theta(1) \text{ if } n = 1; \\ 2T(n/2) + \Theta(n) \text{ if } n > 1. \end{cases}$$

- We shall usually omit stating the base case when $T(n) = \Theta(1)$ for sufficiently small n, but only when it has no effect on the asymptotic solution to the recurrence.
- CLRS and Lecture 2 provide several ways to find a good upper bound on $T(n)$.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

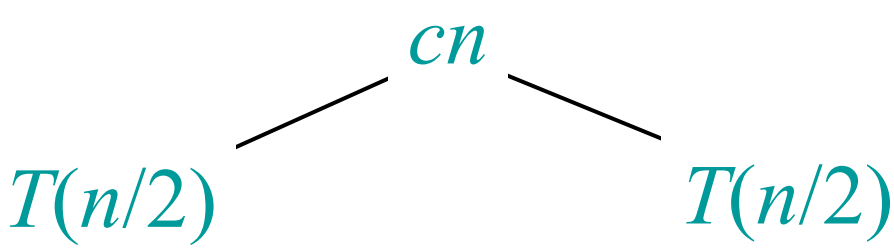
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.

$$T(n)$$

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

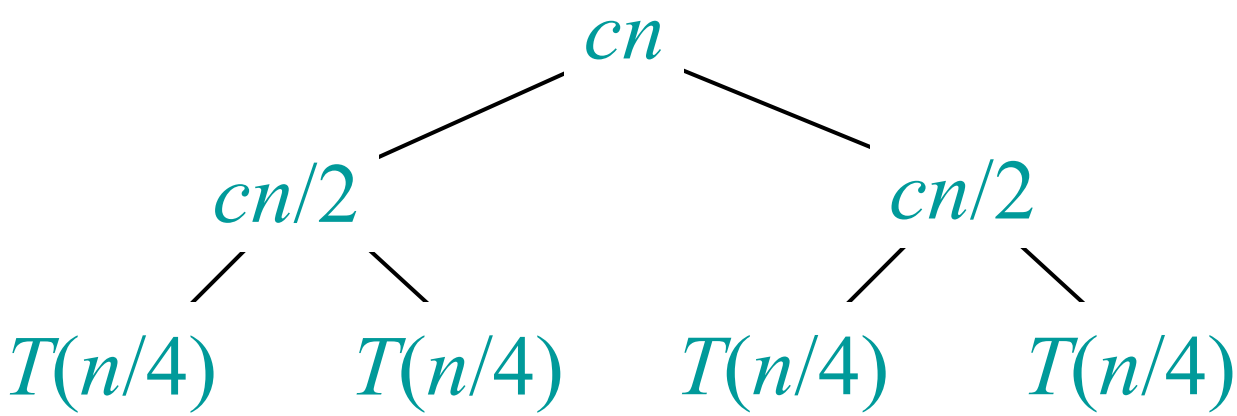
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

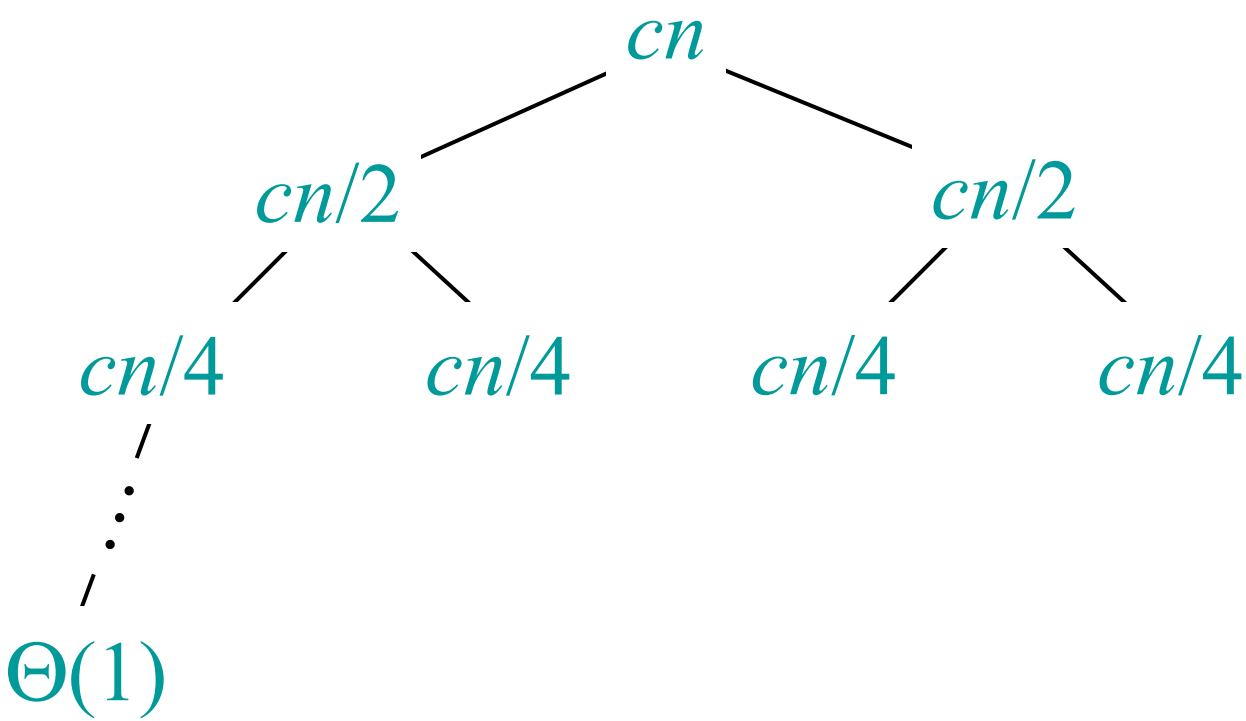
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

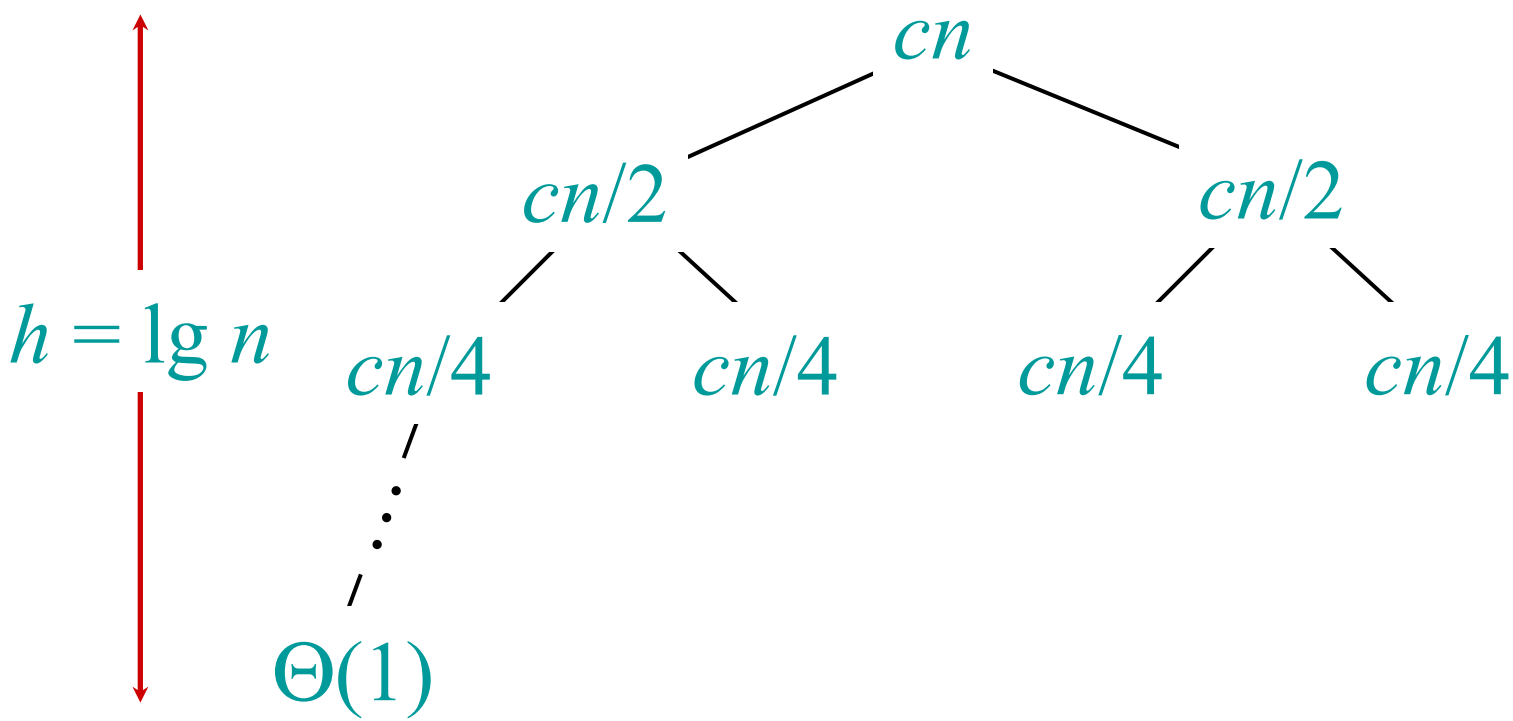
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

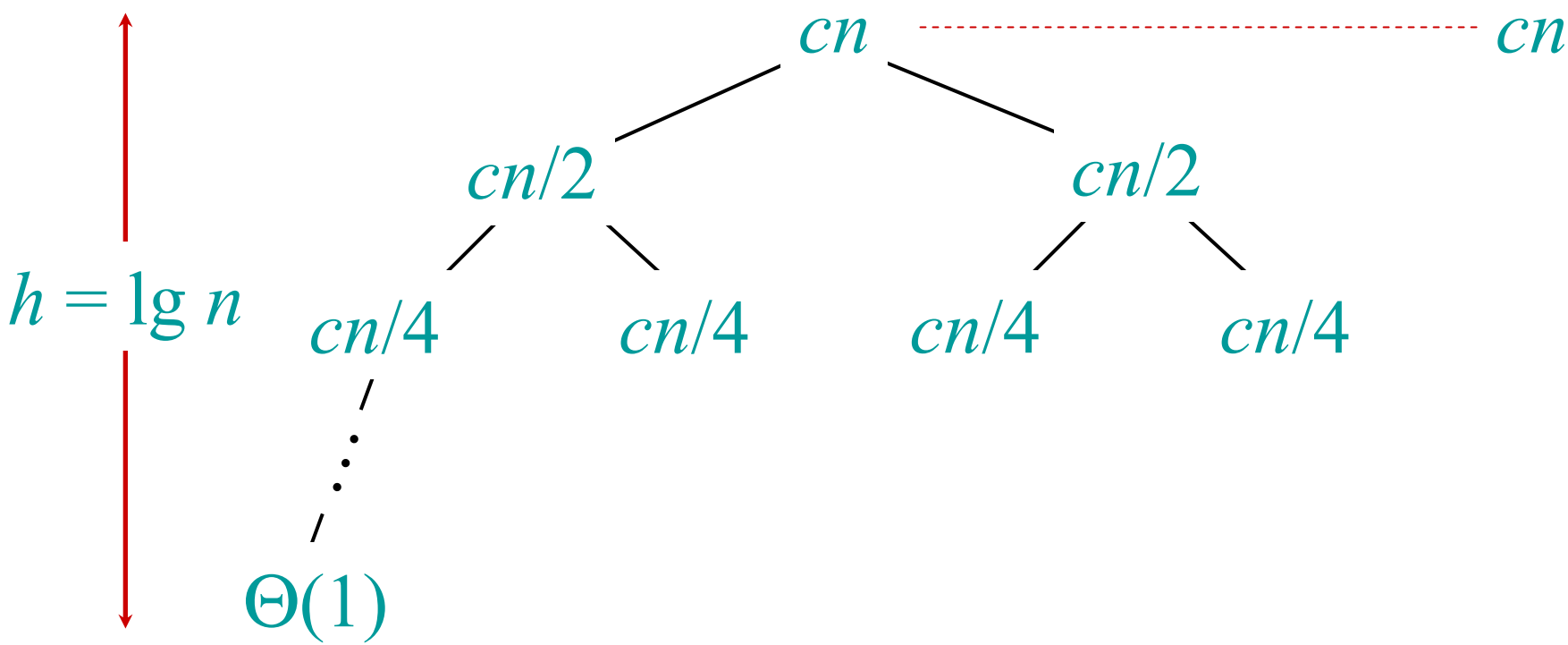
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

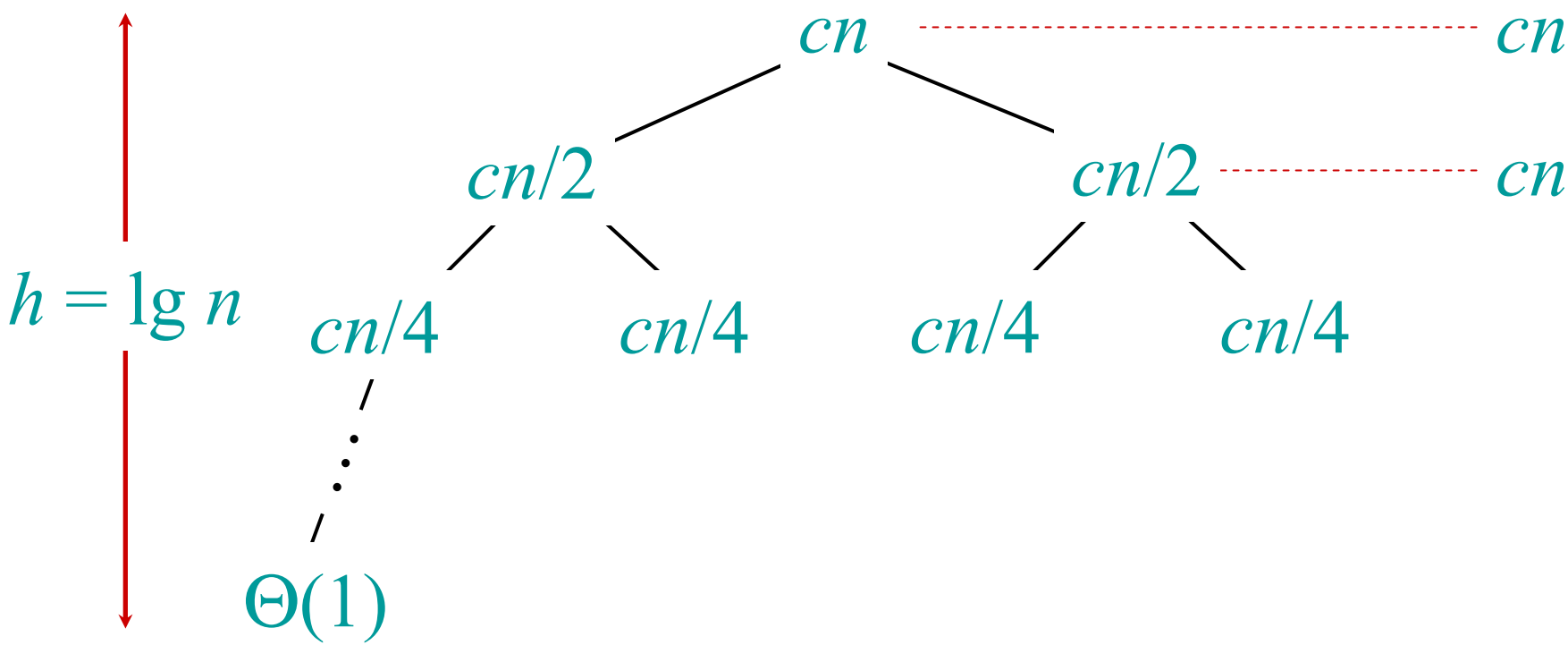
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

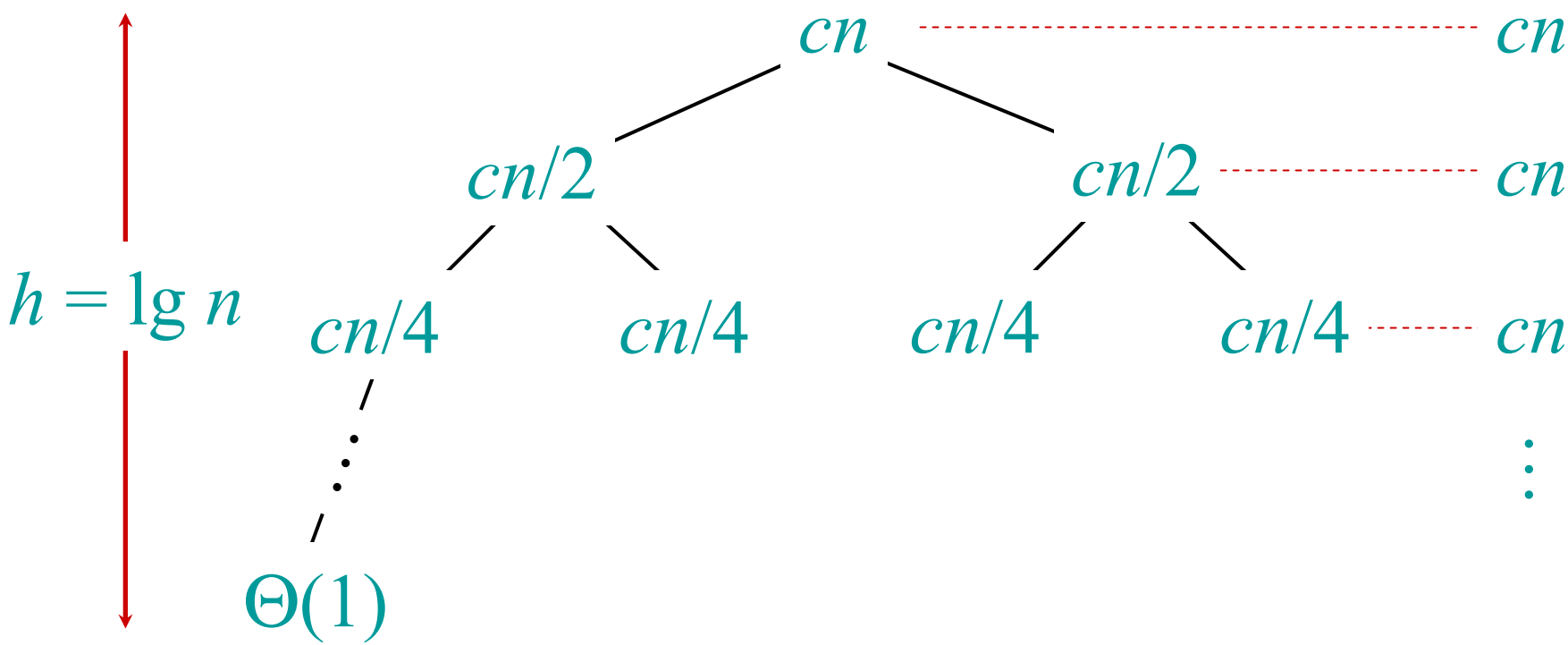
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

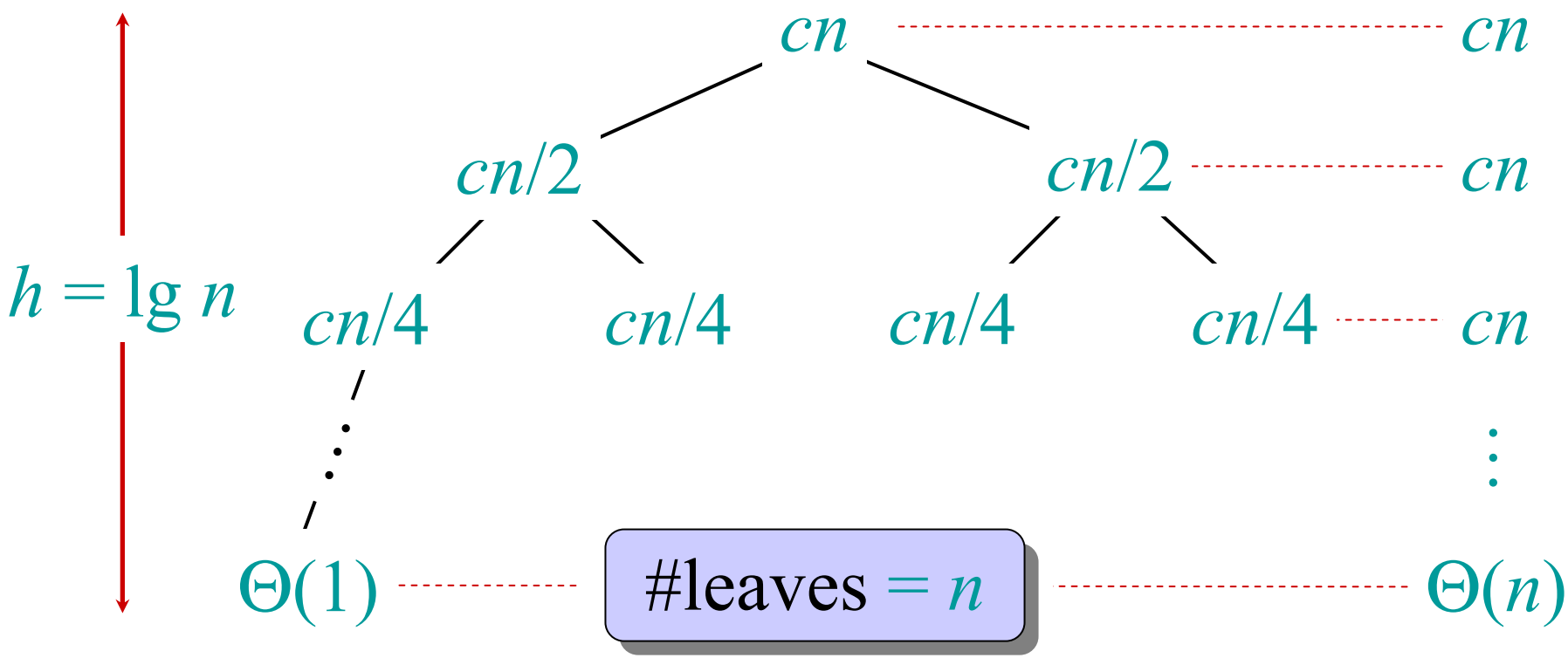
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

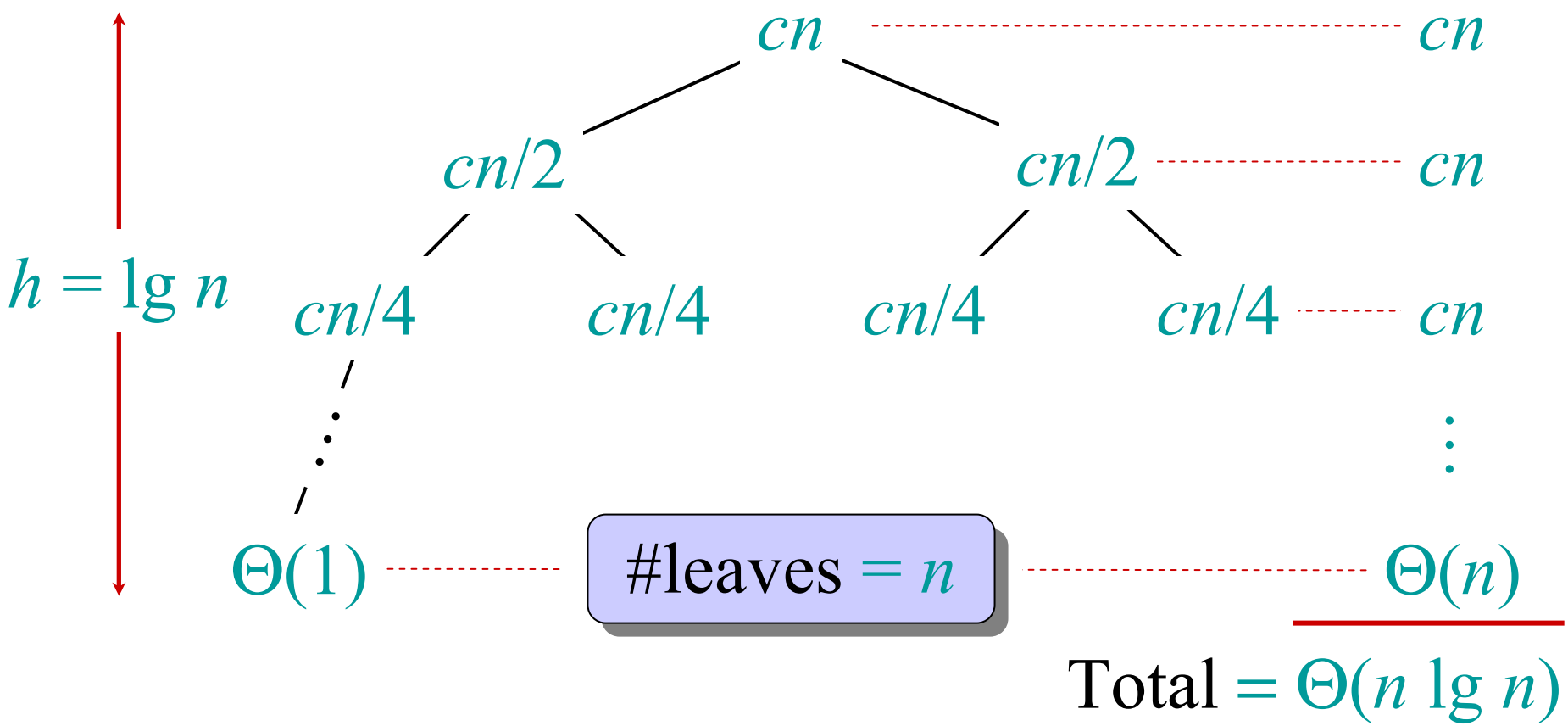
Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Recursion tree

Solve $T(n) = 2T(n/2) + cn$, where $c > 0$ is constant.



Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson

Conclusions

- $\Theta(n \lg n)$ grows more slowly than $\Theta(n^2)$.
- Therefore, merge sort asymptotically beats insertion sort in the worst case.
- In practice, merge sort beats insertion sort for $n > 30$ or so.
- Go test it out for yourself!

Copyright © 2001-5 Erik D. Demaine and Charles E. Leiserson